

Execução de Contêineres do Windows no Compute Engine

1.

Conexão via RDP na VM do Windows

O primeiro passo para rodar contêineres Windows no Google Cloud é configurar uma VM do Windows no Google Compute Engine e conectar-se a ela via RDP (Remote Desktop Protocol).

Passos para Conectar-se via RDP:

1. Criar uma VM Windows no Google Cloud:

- No Google Cloud Console, vá para o Compute Engine e clique em “Criar Instância”.
- Escolha uma imagem do Windows Server (exemplo: Windows Server 2019) como o sistema operacional da VM.
- Defina os recursos como CPU, memória, e rede conforme necessário.
- Habilite a conexão via RDP.

2. Acessar a VM via RDP:

- Após a VM ser provisionada, no Console, clique em “Conectar” e selecione a opção RDP.
- Insira suas credenciais para acessar o servidor.

3. Verificação de Conexão:

- Uma vez conectado via RDP, verifique se a VM está funcionando corretamente. Abra o Gerenciador de Servidores para revisar as configurações básicas.

Você está configurando um servidor Windows para rodar aplicações em contêineres. A conexão via RDP é essencial para gerenciar o sistema operacional e realizar as configurações necessárias para hospedar os contêineres.

2. Criação de um Aplicativo Contêiner do Windows

Agora que você está conectado à sua VM do Windows, o próximo passo é criar um aplicativo que será empacotado e executado em um contêiner Docker no Windows.

Passos para Criar o Aplicativo:

1. Configurar o Ambiente de Desenvolvimento:

- Certifique-se de que o Docker para Windows está instalado e em execução na VM. Você pode baixar e instalar o Docker diretamente do site oficial.

2. Criar um Aplicativo Simples:

Para este exemplo, vamos criar um aplicativo básico em [ASP.NET](#) Core. Abra o Visual Studio ou outro editor de código e crie um projeto:

```
dotnet new web -n MeuAppWindows  
cd MeuAppWindows
```

3. Testar Localmente:

- Teste o aplicativo executando-o diretamente na VM:

```
dotnet run
```

4. Preparar o Aplicativo para Docker:

Crie um arquivo Dockerfile para definir como o aplicativo será empacotado em um contêiner Docker:

```
FROM mcr.microsoft.com/dotnet/aspnet:5.0 AS base
```

WORKDIR /app

EXPOSE 80

FROM mcr.microsoft.com/dotnet/sdk:5.0 AS build

WORKDIR /src

COPY . .

RUN dotnet restore

RUN dotnet publish -c Release -o /app

FROM base AS final

WORKDIR /app

COPY --from=build /app .

ENTRYPOINT ["dotnet", "MeuAppWindows.dll"]

Um desenvolvedor precisa criar um aplicativo para rodar em um contêiner Windows. Usar o [ASP.NET](#) Core para criar um web app é uma escolha comum para desenvolvimento de aplicações em contêineres Windows.

3. Construção de Imagem Docker

Depois de criar o aplicativo, o próximo passo é construir a imagem Docker a partir do Dockerfile.

Passos para Construir a Imagem Docker:

1. Navegar até o Diretório do Projeto:

- No terminal da VM, vá até o diretório do projeto que contém o arquivo Dockerfile.

2. Construir a Imagem Docker:

- Use o comando `docker build` para construir a imagem Docker a partir do Dockerfile:

```
docker build -t meu-app-windows .
```

3. Verificar a Imagem Criada:

- Use o comando `docker images` para listar as imagens criadas e confirmar que a imagem foi gerada corretamente:

```
docker images
```

Depois de desenvolver o aplicativo, a criação de uma imagem Docker permite que você encapsule todas as dependências e a lógica de execução em uma única unidade portátil.

4. Execução do Contêiner do Windows

Agora que a imagem Docker foi construída, o próximo passo é executar o contêiner no ambiente Windows da VM.

Passos para Executar o Contêiner:

1. Executar o Contêiner:

- Use o comando `docker run` para iniciar o contêiner:

```
docker run -d -p 8080:80 meu-app-windows
```

2. Verificar a Execução:

- Use o comando `docker ps` para verificar se o contêiner está rodando:

```
docker ps
```

3. Acessar o Aplicativo:

- Acesse o aplicativo no navegador da VM ou na rede, usando o endereço da VM e a porta configurada (por exemplo, `http://[IP_DA_VM]:8080`).

Após empacotar o aplicativo **ASP.NET** em um contêiner Docker, executá-lo permite que você faça o deploy da aplicação com consistência em qualquer ambiente que suporte Docker.

5. Limpeza e Finalização

Após testar o contêiner, é uma boa prática limpar os recursos que não serão mais usados para evitar custos desnecessários.

Passos para Limpeza:

1. Parar o Contêiner:

Use o comando `docker stop` para parar o contêiner:

```
docker stop [ID_DO_CONTÊINER]
```

2. Remover o Contêiner:

- Para remover o contêiner:

```
docker rm [ID_DO_CONTÊINER]
```

3. Remover a Imagem:

Para remover a imagem Docker criada:

```
docker rmi meu-app-windows
```

4. Excluir a VM do Compute Engine:

- No Google Cloud Console, navegue até o Compute Engine, selecione a VM e clique em “Excluir”.

Limpar os recursos desnecessários, como contêineres e imagens Docker, além de encerrar a VM quando não for mais necessária, ajuda a reduzir custos e otimizar o uso de recursos no Google Cloud.

Conteúdo Bônus

Título: Contêineres no Compute Engine

Plataforma: Cloud.Google

Descrição: “Este documento descreve algumas das tecnologias de contêineres mais comuns que você pode usar para executar contêineres em instâncias do Compute Engine. É possível usar essas tecnologias na maioria das imagens de VM públicas fornecidas pelo Compute Engine.”

Referências Bibliográficas

BASSO, D. E. **Administração de Redes de Computadores**. Contentus, 2020.

KUROSE, J. F.; ROSS, K. W. **Redes de Computadores e a Internet: Uma Abordagem Top-Down**. 8. ed. Pearson, 2021.

MARINHO, A. L.; CRUZ, J. L. da. (Orgs.). **Desenvolvimento de Aplicações para Internet**. 2. ed. Pearson, 2020.

PUGA, S.; RISSETTI, G. **Lógica de Programação e Estruturas de Dados, com Aplicações em Java**. 3. ed. Pearson, 2016.

ROHLING, L. J. **Segurança de Redes de Computadores**. Contentus, 2020.

SILVA, C. F. da. **Projeto Estruturado e Gerência de Redes**. Contentus, 2020.

TANENBAUM, A. S.; FEAMSTER, N.; WETHERALL, D. J. **Redes de Computadores**. 6. ed. Pearson, 2021.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. **Sistemas Digitais: Princípios e Aplicações**. 12. ed. Pearson, 2018.